Final Project Summary

Multimodal Behavioral Cloning for a Robotic Arm Using Mamba and OpenCLIP

Oria Cohen | Computer Science Final Project

Project Type	Offline multimodal robotic imitation learning pipeline
Main Task	Behavioral cloning for continuous next-action prediction
Core Technologies	YOLO, OpenCLIP, Mamba, PyTorch, Streamlit
Dataset Focus	Recorded BridgeData-style robotic manipulation trajectories, including the scripted_6_18.zip subset of approximately 30GB

1. Final Product Description

The final product of this project is a software system for multimodal robotic imitation learning. The system operates on recorded robotic demonstrations and combines visual information from image sequences with robot-state data in order to predict the next robotic action.

The system implements a research and engineering pipeline that includes dataset selection, YOLO-based data curation, filtered CSV generation, OpenCLIP visual embedding extraction, multimodal data fusion, temporal sequence construction, Mamba-based action prediction, model evaluation, and results visualization through a Streamlit user interface.

In the final version, the central model task is behavioral cloning. The model receives a temporal sequence of previous observations and predicts a continuous 7-dimensional robot action vector that represents the next robot action.

The system is not presented as a real-time physical robot control system. It is an offline learning, training, evaluation, and demonstration pipeline that works on recorded trajectories.

2. Implemented System Components

The project implements a complete system that combines data processing, machine learning components, training and evaluation infrastructure, and a user interface for operating the pipeline and reviewing results.

2.1 Dataset Loading and Validation

A dataset selection and validation mechanism was implemented. The system allows the user to select a dataset folder, checks that the selected structure is compatible with the pipeline, identifies trajectory folders, validates required artifacts, and prevents progress to later stages when the selected data is invalid or incomplete.

The system also supports detection of an existing filtered CSV file, validation of required columns, and reuse of a valid CSV artifact. This enables efficient work without rerunning stages that were already completed.

2.2 YOLO-Based Data Curation and Filtering

A significant YOLO-based data curation component was added during the project in order to improve the quality of the dataset before the OpenCLIP, training, and evaluation stages.

This work included more than using an existing detector. Relevant images were collected and organized into groups, annotated manually using Roboflow, and labeled with bounding boxes around both the robot gripper and task objects. These annotated examples were then used to train/fine-tune a YOLO model so that its weights would better match the visual environment and objects appearing in the project dataset.

After obtaining suitable YOLO weights, the model was integrated into the filtering pipeline. Several detection thresholds and filtering conditions were adjusted in order to produce a reliable filtered CSV file. The original dataset contained approximately 8,000 trajectory folders, and after applying the filtering logic, approximately 4,100 trajectories were retained as successful examples for the downstream pipeline.

To validate the filtering behavior, a manual inspection process was performed on a representative sample of trajectory folders. This included checking approximately 80 folders classified as successful and approximately 80 folders classified as failed, in order to verify that the filtering decisions were reasonable and consistent with the visual trajectory behavior.

The filtering policy was intentionally conservative. In ambiguous cases, it was preferable to reject a trajectory that may have been successful rather than incorrectly accept a visually invalid or failed trajectory as successful. This decision was made because the downstream Mamba model learns from the filtered data, and noisy or incorrectly accepted examples could reduce the quality of the behavioral cloning training process.

YOLO is therefore used as an important data curation component together with robot-state checks. Its role is to reduce silent failures, where robot-state signals alone may not fully indicate whether a recorded demonstration is visually valid.

2.3 Filtered CSV Generation

After the filtering stage, the system generates or detects a filtered CSV file that contains the trajectory list and their status. This CSV is used as the basis for the following stages: embedding generation, sample loading, training, and evaluation.

The system validates that the CSV contains the required columns and displays useful information such as the number of examples, the number of valid trajectories, and the file update time.

2.4 OpenCLIP Visual Embedding Generation

An OpenCLIP embedding stage was implemented. This stage converts trajectory images into visual embedding vectors. Each image is encoded into a 512-dimensional feature vector that represents the visual content in a more compact and stable way than raw pixels.

To improve the training workflow, embeddings are precomputed and stored as cache files. This separates the expensive visual processing stage from the Mamba training loop and avoids recomputing OpenCLIP embeddings during every epoch.

The system supports detection of existing embeddings, cache reuse, recomputation when requested, and identification of partial embedding artifacts.

2.5 Multimodal Fusion of Visual and Robot-State Data

A fusion mechanism was implemented to combine the visual embedding with the robot-state vector at each timestep. The 512-dimensional visual embedding is concatenated with a 7-dimensional robot-state vector, producing a unified 519-dimensional multimodal representation.

This representation gives the model both the visual context of the scene and the kinematic state of the robot, instead of relying on only one source of information.

2.6 Sequence Construction for Training and Evaluation

A DataLoader was implemented to load the filtered CSV, embedding files, and robot-state data, and to construct fixed-length temporal windows. Each temporal window is used as an input sequence for the Mamba model.

The system focuses on predicting a future action from a previous sequence. Therefore, each input window is paired with a target action from the demonstration data. This implements the behavioral cloning principle: the model learns to imitate the actions performed in the recorded robotic demonstrations.

2.7 Mamba Model for Action Prediction

A Mamba-based model was implemented to receive a sequence of multimodal vectors and predict the next robot action. Mamba was selected because it is designed for efficient sequence modeling while preserving temporal context across previous timesteps.

The model includes an input projection layer, a Mamba sequence-processing layer, and an output head that predicts a 7-dimensional robot action vector.

2.8 Training Workflow

A complete training workflow was implemented. It includes:

- Loading the filtered dataset.
- Using precomputed embeddings.
- Configuring hyperparameters.
- Splitting data into train and validation sets.
- Computing loss.
- Saving checkpoints.
- Saving the run configuration.
- Saving metrics.
- Applying early stopping based on patience.
- Writing logs for tracking and reproducibility.

This workflow makes it possible to run different experiments and store their artifacts in an organized checkpoint structure.

2.9 Checkpoint and Weights Management

The system includes a mechanism for detecting existing checkpoints, displaying basic information about saved runs, and selecting an appropriate checkpoint for evaluation using weights that match the executed training and evaluation runs.

In addition, the system prevents evaluation when no checkpoint is selected or when the required weights file is missing.

2.10 Evaluation

An evaluation stage was implemented. It loads a trained checkpoint, runs the model on the selected data, computes error metrics, and produces prediction examples for review.

The main metrics and outputs include:

- MSE.
- MAE.
- Per-dimension MAE.
- Number of evaluated samples.

- Prediction examples compared to ground truth.

The evaluation stage allows both quantitative assessment of prediction quality and qualitative inspection of prediction examples.

2.11 Streamlit User Interface

A full Streamlit interface was developed to organize and operate the complete workflow. The UI includes the following stages:

- Dataset screen.
- YOLO screen.
- OpenCLIP screen.
- Model / Training screen.
- Evaluation screen.
- Results screen.

The interface displays progress, logs, stage status, existing artifacts, reuse and recompute options, checkpoint selection, evaluation results, charts, tables, and image examples.

The system also stores process and stage state, so that the pipeline status can be understood after navigating between screens or reopening the interface.

2.12 Results Visualization

The Results screen presents the evaluation outputs in a concentrated and interpretable way. It includes:

- MAE and MSE metrics.
- Per-dimension error chart.
- Prediction versus ground-truth chart.
- Example image from a trajectory.
- Navigation between prediction examples.
- Action comparison table.
- Evaluation log.

The purpose of this screen is to support both numerical and visual understanding of the model prediction quality.

3. Scope Changes During the Project

During the project, the scope became more focused as the dataset structure, trajectory format, and engineering complexity of a full pipeline became clearer.

At the beginning, the project was defined more broadly around visual sequence learning in robotics, with several possible output types such as success/failure prediction, future-state prediction, or action prediction. After implementation progressed and the dataset structure was analyzed, the system focused on a clearer and more concrete task: continuous robot action prediction using behavioral cloning.

As a result, the central output of the final model is a continuous 7-dimensional action vector rather than a binary success/failure classification. However, success/failure information remains meaningful in the data filtering and curation process.

Another significant change was the addition of YOLO as part of the data filtering and curation process. Early design documents included the general concept of data filtering and improving data quality, but during implementation this was realized through YOLO and visual checks in order to handle silent failures and examples where robot-state signals alone were not sufficient.

The user interface also evolved during the project. The initial direction was to provide a basic execution and results interface, while the final system includes a broader UI that controls pipeline stages, shows logs and progress, supports checkpoint selection, runs evaluation, and displays a full results view.

4. Items Outside the Final Project Scope

The following items were not included in the final implementation scope or were outside the central focus of the project.

4.1 Real-Time Physical Robot Control

The system does not deploy the learned model on a physical robot and does not send commands in real time to a robotic arm. The project focuses on learning and evaluation using recorded data, and is therefore not presented as a real-time closed-loop control system.

4.2 Success/Failure Prediction as the Main Mamba Output

Earlier project documents mentioned the possibility of supporting binary success/failure sequence classification. In the final implementation, the central Mamba task focuses on continuous action prediction. Success/failure information is used mainly for filtering, data curation, and selecting higher-quality examples for training.

The choice to focus on action prediction is more aligned with behavioral cloning and with the dataset structure, which includes real action vectors from recorded demonstrations.

4.3 OpenCLIP Fine-Tuning

OpenCLIP is used as a frozen pretrained visual encoder. The project does not fine-tune OpenCLIP on the robotic dataset. This design keeps the visual representation stable, reduces computational cost, and leaves the main learning task to the Mamba sequence model.

4.4 Full Experimental Comparison with Alternative Models

The literature review and design process considered alternatives such as RNN, LSTM, and Transformer models. However, the final project focuses on implementing a working Mamba-based pipeline rather than performing a full experimental benchmark against all alternative architectures.

The comparison between approaches was used to justify the architectural choice and define the research direction, while the implementation focused on the Mamba + OpenCLIP system.

4.5 Full Evaluation on All BridgeData Tasks

The system was developed and tested on a large and relevant subset of BridgeData, including a dataset subset of approximately 30GB. The project was not intended to benchmark every BridgeData version and every task type. Instead, it focuses on the selected data used to build a stable and demonstrable pipeline.

4.6 Deep Compatibility Analysis for Every Historical Checkpoint

The system supports checkpoint detection, checkpoint selection, display of basic checkpoint information, and use of relevant weights for evaluation. It does not include a separate system that performs a deep compatibility analysis for every historical checkpoint against every possible architecture change.

In practice, evaluation is performed using relevant and up-to-date checkpoints that match the training and evaluation runs used in the project.

4.7 User Authentication and Access Control

The system is intended to run locally as a research and experimentation tool. Therefore, authentication, authorization, and user-management mechanisms were not part of the project scope.

5. System Testing

System testing is documented in the STD document, which is based on the STP. The goal of the tests is to verify that the pipeline stages work together consistently and that the workflow can proceed from dataset selection to model results visualization.

The tests cover the following main areas:

- Dataset selection and validation.
- YOLO filtering and CSV generation/reuse.
- OpenCLIP embedding generation and cache handling.
- Data preparation and sequence construction.
- Model training and checkpoint handling.
- Evaluation and metric calculation.
- Results visualization.
- UI flow, logs, state persistence, and error handling.

The tests are not limited to model quality. They also verify the engineering quality of the system, including missing-file handling, artifact detection, prevention of invalid stage transitions, log display, artifact saving, and clear results presentation.

6. Known Limitations

The system depends on a GPU/CUDA environment for efficient execution of OpenCLIP, Mamba, and model training. Running the full workflow on CPU is theoretically possible, but is expected to be significantly slower.

Prediction quality depends on the quality of the filtered data, the quality of the visual embeddings, and the checkpoint selected for evaluation. For this reason, YOLO-based data curation, precomputed embeddings, and checkpoint management are important parts of the system.

The system focuses on offline next-action prediction. Therefore, it should not be interpreted as a complete real-time physical robot control solution or as an automatically generalizable system for all BridgeData tasks.

7. Conclusion

The project evolved from an initial research direction on visual sequence learning in robotics into a complete software system that implements a multimodal pipeline for robotic action prediction.

The final implementation combines OpenCLIP for visual representation, robot-state data, YOLO for data filtering and quality improvement, Mamba for sequence learning, and a Streamlit interface for operating the pipeline and presenting the results.

Although the scope became more focused during the project, these changes were based on a better understanding of the dataset and the system requirements. The final result is a complete, modular, and clear engineering system that supports end-to-end robotic imitation learning on recorded data, produces measurable evaluation outputs, and presents the results in an accessible and organized way.